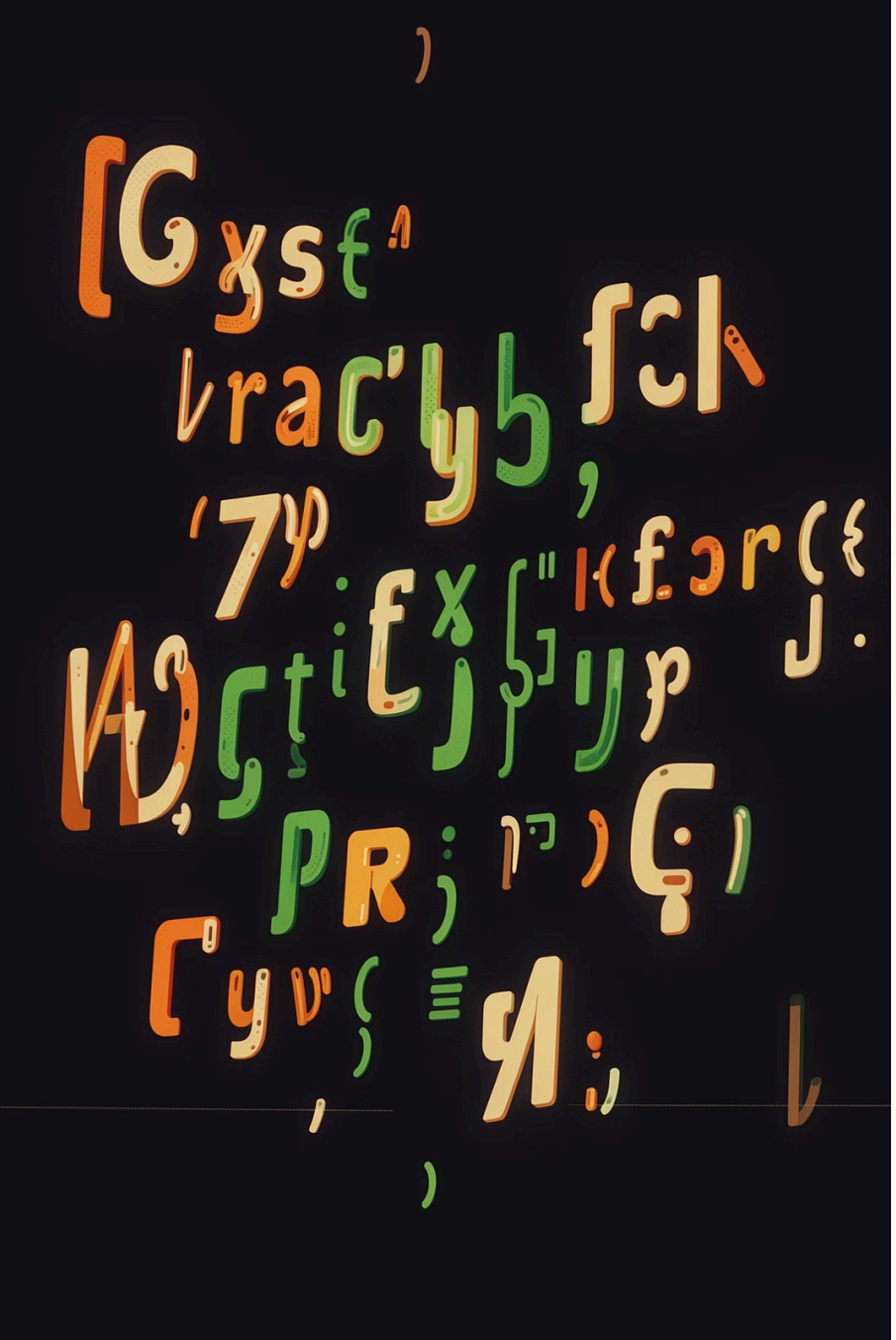


Python Strings: The Building Blocks of Textual Data

From simple words to complex data — strings are at the heart of every Python program.

Made with GAMMA

What is a String in Python?

A **string** is a sequence of characters — letters, symbols, spaces — stored as Python's built-in `str` data type. Strings can represent anything from a single letter to an entire document.

Single Quotes

`'Hello'`

Double Quotes

`"Hello"`

Triple Quotes

`"""Hello"""`

Made with GAMMA

Creating Strings: Your First Textual Data

String Literals

The most direct way — simply type text within quotes and assign it to a variable.

```
message = 'Hello, Python!'
greeting = "Welcome!"
```

Using `str()` Function

Convert other data types — integers, floats, booleans — into string form for display or processing.

```
number = 123
text_number = str(number)
# text_number is now "123"
```

Made with GAMMA

The Immutable Nature of Strings

Once created, a string **cannot be changed**. Any operation that appears to modify a string actually creates a brand-new string object in memory.

Think of strings like a carved stone tablet — you can't alter the original carving. Instead, you chisel a new one. This immutability makes strings **safe, predictable, and hashable** in Python.

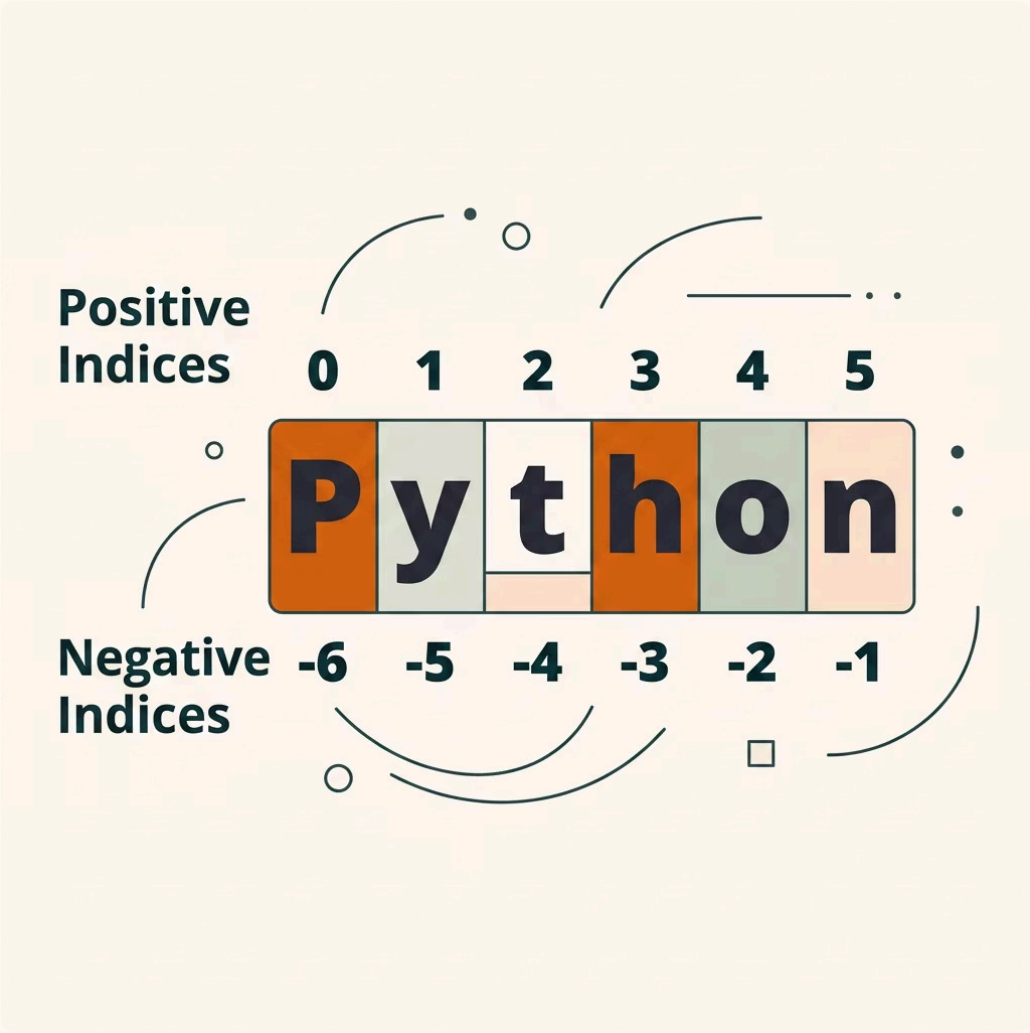
Made with GAMMA

Accessing Characters: Peeking Inside Strings

Index-Based Access

Use square brackets `[]` with a zero-based index to retrieve any character in a string.

```
my_string = "Python"
my_string[0]   # 'P'
my_string[2]   # 't'
my_string[-1]  # 'n' (last char)
```



Made with GAMMA

Slicing Strings: Extracting Substrings

Use the `[start:end]` syntax to extract a portion of a string. The `start` index is inclusive, while `end` is exclusive.

1

Full Range

`s = "Programming"`

`s[0:4]` → **"Prog"**

2

From Index

`s[4:]` → **"ramming"**

From index 4 to the end

3

Up to Index

`s[:4]` → **"Prog"**

From start up to index 4

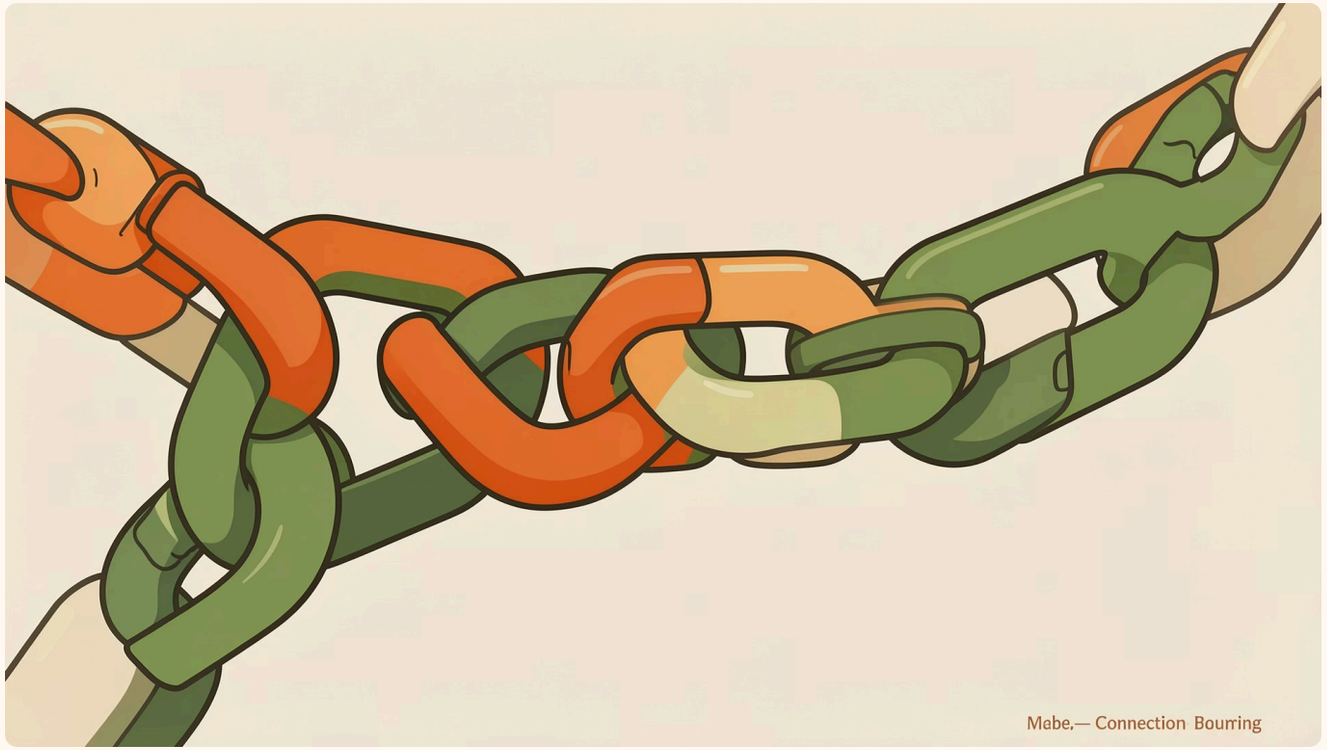
Made with GAMMA

Combining Strings: Concatenation

Join strings together using the `+` operator — perfect for building dynamic messages and names.

```
first_name = "Ada"
last_name = "Lovelace"
full_name = first_name + " " + last_name
# Output: "Ada Lovelace"
```

For joining **many strings** efficiently, use `", ".join(list_of_strings)` — it's faster and cleaner than repeated `+` operations.



Made with GAMMA

Powerful String Methods: Beyond the Basics

.upper() / .lower()

Change the case of all characters: `"hello".upper()` → `"HELLO"`

.find()

Locate a substring's starting index: `"Python".find("th")` → `2`

.replace()

Substitute parts of a string: `"old".replace("o", "new")` → `"newld"`

Formatting Strings: Dynamic Text

f-Strings [1]

The modern, readable way to embed variables and expressions directly inside string literals.

```
name = "Alan Turing"
age = 41
bio = f"My name is {name} and I was {age} years old."
```

.format() Method

A powerful alternative compatible with Python 2 and older Python 3 codebases.

```
bio = "My name is {} and I was {} years old.".format(name, age)
```

ⓘ f-strings are faster and more readable — prefer them in all modern Python 3.6+ projects.

Made with GAMMA

Strings: Your Gateway to Textual Power in Python

User Input

Capture and process what users type

File Handling

Read, write, and parse text files

Data Processing

Parse, transform, and analyze raw data

APIs & Web

Send and receive text-based responses

Mastering strings is not just a first step — it's a **foundation** that unlocks the full power of Python programming.

Made with GAMMA